

O caminho dos bytes no Checkpoint 2

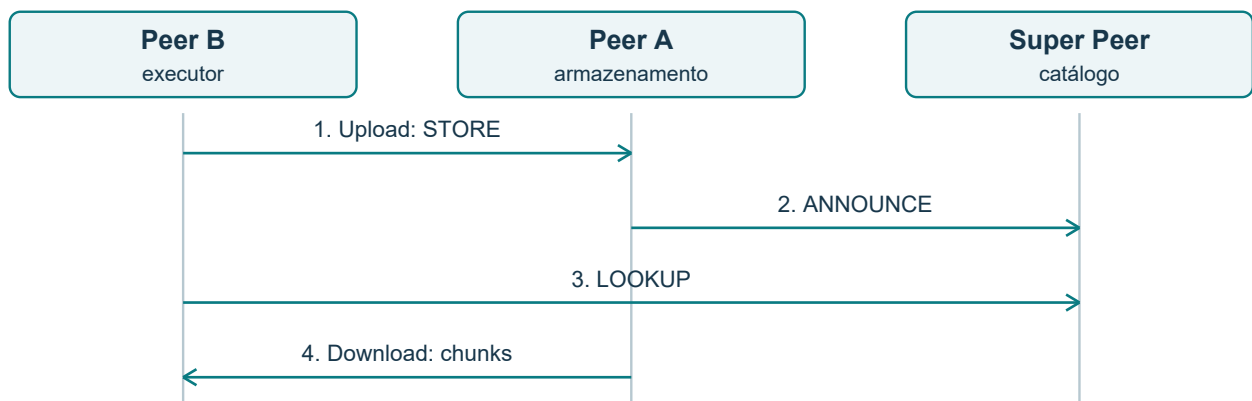
Material de estudo e apresentação, revisado em 29/09/2026 a partir dos fontes atuais. Leitura da versão Git 9dbe2c5.

No C1, os processos aprenderam a trocar mensagens TCP. No C2, essa comunicação passa a transportar documentos: identificar, dividir, comprimir, enviar, armazenar e reconstruir o conteúdo. A parte do Aluno 1 coordena esse caminho e utiliza o catálogo integrado com a parte do Aluno 2.

Componente	Responsabilidade
Peer	Guarda chunks e atende pedidos. Também pode solicitar uploads e downloads a outros nós.
Super Peer	Mantém metadados e localizações dos chunks para responder às consultas.
CLI local	Encaminha o comando ao Peer ativo e mostra o progresso e o resultado.

Um exemplo que acompanha o guia

O Peer B executa um upload para o Peer A. A recebe os chunks e anuncia o objeto ao Super Peer. Depois B consulta o catálogo e baixa de A. As portas do exemplo são 55103 para B, 55102 para A e 55101 para o Super Peer.



A identidade acompanha o serviço

A CLI envia a operação por um socket Unix local. O serviço cria a sessão de transferência com seu NodeID, já utilizado no JOIN. O parâmetro `--local-peer-port` escolhe esse executor; host e porta remotos escolhem o destino da operação.

Para apresentar: o Super Peer informa onde estão as partes. Os Peers mantêm e transferem os bytes diretamente entre si.

Partes do arquivo, hashes e compressão

Chunk é uma região de bytes do documento. Índice, offset e tamanho permitem identificar e remontar cada região.

Exemplo: documento com 10 MiB

METADATA_CHUNK_SIZE vale 4.194.304 bytes, ou 4 MiB. Um documento de 10 MiB gera três chunks; o último contém somente o restante. As partes não precisam ser PDFs válidos separadamente.

Índice	Offset em bytes	Tamanho original
0	0	4 MiB
1	4.194.304	4 MiB
2	8.388.608	2 MiB

O índice começa em zero. O offset do chunk é índice multiplicado por 4 MiB. Se o chunk 2 chegar antes do 0, seu offset continua indicando a posição correta no arquivo de destino.

ObjectID, NodeID e TransactionID

ObjectID é o SHA-256 do arquivo inteiro, calculado em leitura incremental de blocos. **NodeID** identifica o nó e deriva de IP binário, porta e UUID. **TransactionID** associa pedido e resposta. Os dois primeiros têm 32 bytes; o último tem 16. Um hash de 32 bytes aparece no terminal como 64 dígitos hexadecimais.

A ordem dentro de um chunk

O worker lê os bytes originais, calcula SHA-256 e comprime com LZ4. O receptor descomprime e calcula o hash novamente. O hash representa o conteúdo original, enquanto o transporte leva os bytes comprimidos. A LZ4 processa cada chunk de forma independente.

O tamanho comprimido pode ser maior que o original. compression.c usa o limite seguro indicado pela biblioteca para reservar saída. Arquivos que já contêm dados comprimidos podem ter pouco ganho; o benchmark informa o desempenho observado na máquina.

Verificações em níveis diferentes

O CRC32 da zlib verifica header e payload serializados. O SHA-256 do chunk verifica a parte descomprimida. O ObjectID verifica o documento completo reconstruído. O programa usa OpenSSL para SHA-256 e a biblioteca externa LZ4 para compressão; esses algoritmos não foram reimplementados.

Para apresentar: identificar e verificar cada parte permite trabalho independente; a conferência do arquivo inteiro confirma o resultado final. Esses hashes não fornecem autenticação do remetente.

Upload: do BEGIN ao anúncio

file_client.c prepara e coordena a operação; o serviço incorporado a peer.c recebe os pedidos e utiliza storage.c.

1. Preparar o arquivo e descobrir o destino

O Peer executor verifica a extensão .pdf, sem distinguir maiúsculas, e tenta abrir o arquivo. Calcula ObjectID e tamanho; arquivo vazio não conclui upload. A validação é pela extensão, sem parser de PDF. O arquivo não deve mudar durante a leitura. Um PING inicial descobre o NodeID do armazenamento para conferir as operações seguintes.

2. STORE / BEGIN

O pedido informa ObjectID, nome, tamanho, quantidade de chunks e LZ4. O receptor prepara um registro e manifest em pending ou aceita um upload anterior compatível. O ACK confirma essa preparação, ainda sem afirmar que o documento está completo.

3. STORE / CHUNK em paralelo

Workers reservam índices de trabalho e usam pread para ler a região correspondente. Cada pedido contém índice, offset, tamanhos original/comprimido, hash e bytes LZ4. Os workers abrem conexões próprias; o receptor responde ACK ou ERROR para cada parte.

A mensagem primeiro passa pelo framing e CRC32. Depois o armazenamento descomprime, compara SHA-256 e confere se índice, offset e tamanho correspondem ao documento. Uma parte duplicada só é aceita se seus descritores forem compatíveis; divergência retorna erro.

4. STORE / COMMIT

Após aguardar os workers, o executor pede a conclusão. O receptor exige todos os chunks, reabre e descomprime em ordem, verifica hashes e atualiza um SHA-256 incremental do documento inteiro. Confere tamanho e ObjectID antes de publicar o diretório em objects.

5. STORE / ANNOUNCE ao Super Peer

O Peer que armazenou o objeto envia os metadados e todos os descritores ao Super Peer. Esse servidor verifica o cadastro do anunciante e registra o documento com suas disponibilidades. O ACK de COMMIT ao executor só é enviado quando o anúncio também teve sucesso.

No exemplo B para A: os pedidos de upload têm origem B. O anúncio tem origem A, pois A é quem guardou as partes. Se o anúncio falhar, os bytes podem existir localmente e a operação ainda devolver erro; a sequência não implementa 2PC.

Armazenamento e retomada

storage.c mantém os dados locais e controla quando um objeto passa a estar publicado para atender downloads.

Área ou arquivo	O que representa
pending/<ObjectID>	Upload iniciado e ainda não publicado; pode conter partes recebidas.
objects/<ObjectID>	Objeto publicado após confirmação de conteúdo e tamanho.
manifest.bin	Metadados, dono, estado e descritores/presença dos chunks. Formato local versionado.
chunk-<índice>.lz4	Bytes comprimidos de uma parte do documento.
node.uuid	Componente persistente da identidade do serviço.

A raiz padrão é `.peer_storage/<porta>`, relativa ao diretório do serviço, e pode ser configurada por `--data-dir`. Os caminhos internos usam o ObjectID hexadecimal, permitindo manter os bytes separados dos nomes recebidos.

A diferença entre gravar e publicar

Chunks e manifests são escritos em temporários e sincronizados com fsync. A publicação usa rename e sincronização de diretórios. No COMMIT, o documento só recebe FINISHED após a confirmação local; essa publicação antecede o anúncio de rede.

O lock do catálogo permite localizar o registro; um mutex por documento protege suas operações demoradas. Se a persistência do manifest do chunk falhar, os campos anteriores são restaurados em memória. Isso evita aceitar uma repetição como se a gravação malsucedida já estivesse confirmada.

Repetir upload e reiniciar o Peer

Um novo BEGIN com o mesmo ObjectID e dados compatíveis reutiliza o registro existente. Chunks presentes compatíveis são aceitos novamente. Essa idempotência evita duplicar o objeto; a retomada depende de o usuário repetir a operação.

Na inicialização, o Peer carrega manifests, rejeita registros inválidos e verifica o conteúdo dos finalizados. Depois de JOIN, reanuncia os objetos publicados. O UUID persistido permite recuperar o NodeID quando IP e porta permanecem iguais.

Para apresentar: pending descreve trabalho em andamento; objects descreve conteúdo publicado. O manifest permite recuperar o catálogo e conferir as partes. Nenhuma dessas etapas cria réplicas automaticamente.

Metadados: contrato atual entre os alunos

O modelo agora usa FileMetadata, declarado em metadata.h. directory.c adapta os dados recebidos para esse contrato.

Campo de FileMetadata	Significado
object_id[32]	Identificador binário do conteúdo completo.
filename[256]	Nome cadastrado para localizar o documento.
size	Tamanho original em bytes, com 64 bits.
chunk_count	Quantidade de chunks, com 32 bits.
chunk_hashes	Coleção de hashes SHA-256 das partes, mantida na memória do processo.
version	Versão inicial 1; não há versionamento distribuído nesta entrega.
owner	Número de 32 bits derivado dos quatro primeiros bytes do NodeID do anunciante.

Onde fica a identidade completa do proprietário?

O campo owner é resumido para esse modelo. As disponibilidades de chunks continuam guardando o NodeID completo, de 32 bytes. É esse identificador completo que a tabela de membros resolve em IP e porta. O owner numérico não deve ser usado sozinho como endereço ou identidade única de roteamento.

Documento, descritor e localização

TransferDocument descreve o documento no fluxo de transferência e inclui LZ4. MetadataChunk registra índice, offset, tamanhos e hash. As disponibilidades associam o índice ao NodeID de cada Peer. O anúncio leva esses dados serializados, sem transmitir ponteiros como chunk_hashes nem a struct C diretamente.

O anúncio e a consulta

metadata_announce prepara um cadastro completo, valida os descritores e combina localizações antes de publicar a entrada sob mutex. No LOOKUP, directory.c obtém documento e descritores, consulta os NodeIDs disponíveis e usa a tabela de membros de superpeer.c para montar endpoints.

O índice fica na memória do Super Peer. O mesmo ObjectID preserva o primeiro nome; repetir bytes idênticos com outro nome não cria um alias. Se um nome aponta para conteúdos diferentes, a consulta exige ObjectID para resolver a ambiguidade.

Para apresentar: o catálogo descreve e localiza. O armazenamento conserva os bytes. O ponteiro de uma estrutura é válido apenas no processo que o possui; a rede leva uma representação explícita dos dados.

Download: localizar, conferir e remontar

file_client.c coordena a busca. O serviço de peer.c atende cada pedido utilizando os chunks publicados em storage.c.

1. LOOKUP por nome ou ObjectID

O Peer executor descobre o NodeID do Super Peer e consulta o documento. A resposta DOWNLOAD_REQ de metadados inclui descritores e localizações por chunk. Conhecendo os NodeIDs retornados, o executor também confere as identidades esperadas nas respostas dos Peers.

2. Preparar a saída parcial

O programa cria <destino>.part com exclusividade e ajusta seu tamanho. Um destino existente ou .part preexistente não é sobrescrito. Quando o destino é omitido, usa o nome cadastrado no diretório de trabalho da CLI, informado pelo canal local.

3. DOWNLOAD_REQ diretamente ao armazenamento

Cada worker solicita ObjectID e índice do chunk a um endpoint anunciado. O Peer de armazenamento lê os bytes LZ4 e envia DOWNLOAD_REQ de chunk. O executor verifica identidade e correlação, descritor, posição, tamanho, descompressão e hash.

O SHA calculado precisa coincidir tanto com o hash que veio no chunk quanto com o esperado no catálogo. Só então pwrite grava os bytes originais no offset correto. Por isso as respostas podem chegar fora de ordem sem alterar a posição do conteúdo.

4. Fallback entre localizações existentes

Se um endpoint falhar, devolver conteúdo inválido ou não responder no prazo, o worker tenta o próximo anunciado para a parte. Se todos falharem, retorna erro. O fallback utiliza cópias que já foram disponibilizadas; não cria uma nova localização.

5. Confirmar o documento completo

Depois de aguardar as threads, o programa sincroniza o .part, calcula seu SHA-256 e compara tamanho e ObjectID. Usa link para publicar o nome final sem substituir outro destino; sincroniza o diretório, remove o nome .part e sincroniza novamente.

Limite da publicação: uma falha posterior a link pode deixar o destino publicado e ainda produzir erro. Em falha de transferência, a limpeza remove apenas o .part criado pela própria operação. A implementação não oferece retomada automática de download.

Paralelismo, mensagens e controle de erro

A mesma transferência combina workers de chunks, servidor TCP e um protocolo que delimita os bytes recebidos.

Três formas de trabalho

O pool de transferência usa, por padrão, o mínimo entre CPUs, chunks e 8.

PEER_TRANSFER_THREADS aceita de 1 a 32, limitado ao número de partes. O servidor compartilhado tem 32 workers e fila de 64 conexões. O controle Unix atende um comando local por vez, enquanto seus chunks continuam paralelos.

A variável de workers deve estar no ambiente do serviço Peer executor. Alterá-la somente no processo CLI não reconfigura o serviço que já está rodando. Mutexes protegem reserva de índices, primeiro erro e contadores; pread/pwrite usam offsets explícitos e não disputam o cursor do arquivo.

O que acontece quando um worker falha?

O contexto guarda o primeiro erro. Outros workers deixam de reservar novas tarefas, mas as threads já iniciadas são aguardadas antes de liberar o contexto. Cada RPC usa uma conexão própria; o estado de framing não é compartilhado entre workers.

Header, framing e CRC32

O protocolo lê 98 bytes de header, com versão, tipo, origem, destino, TransactionID, timestamp, comprimento e CRC. Depois lê exatamente o payload indicado, limitado a 5 MiB. A margem comporta o pior caso de compressão de um chunk e seu descritor.

TCP entrega um fluxo: send e recv podem tratar apenas parte do pedido. network.c acumula bytes e retoma interrupções. protocol.c valida tamanho, header e CRC antes de entregar a mensagem ao handler. transfer_protocol.c organiza os campos do corpo em ordem de rede.

Prazos, identidade e estados

Os padrões são 5 segundos para conexão e 30 segundos de inatividade de I/O; avanço positivo renova o prazo. RPC confere TransactionID e identidades conhecidas, e a operação confere o tipo da resposta. STORE e LOOKUP C2 exigem origem não nula; isso não é autenticação criptográfica.

Os contextos usam CREATED, QUEUED, STARTED, TRANSFERRING, VERIFYING e FINISHED. O manifest representa o estado local de armazenamento e não registra necessariamente cada passagem desses contextos. REPLICATED permanece reservado.

Para apresentar: paralelismo distribui trabalho; sincronização mantém os índices e estados coerentes; framing separa mensagens; hashes conferem os dados. Essas responsabilidades se complementam.

Mapa atualizado dos arquivos

As responsabilidades foram consolidadas. Os nomes abaixo existem no código atual e na compilação do Makefile.

Arquivo	Como participa do C2
peer.c	Entrada da CLI e serviço de armazenamento: JOIN, recepção de STORE, pedidos de chunks, anúncio e LEAVE.
superpeer.c	Entrada, atendimento TCP e tabela de membros do Super Peer. Integra o catálogo às mensagens.
file_client.c	Coordena upload/download, workers, validação do resultado, progresso e benchmark LZ4.
local_control.c	Canal Unix entre comando local e serviço executor com NodeID ativo.
storage.c	Manifests, chunks, pending/objects, verificação e publicação dos objetos.
compression.c / content.c	LZ4 e auxiliares de hash de buffers, extensão, nome e sincronização.
metadata.c / directory.c	FileMetadata, descritores, disponibilidades e resolução das localizações.
node.c / app_config.c	Identidade, UUID persistente e configuração dos serviços.
transfer_protocol.c	Payloads de BEGIN, CHUNK, COMMIT, ANNOUNCE, LOOKUP e download.
rpc.c / protocol.c / network.c	Correlação de pedido/resposta, framing/CRC32 e transporte TCP.
concurrent_server.c	Pool limitado de atendimento, fila e encerramento das conexões.

O que mudou desde o PDF anterior

peer_service.c foi incorporado a peer.c. membership.c e superpeer_app.c foram incorporados a superpeer.c. Por isso não devem ser apresentados como arquivos separados nesta revisão. FileMetadata substituiu o modelo anterior de metadados do índice.

Executáveis, aliases e testes locais

bin/peer e bin/superpeer têm suas próprias entradas. bin/client aponta para peer e bin/node para superpeer. O teste local de membros compila superpeer.c com SUPERPEER_MEMBERSHIP_ONLY, omitindo o atendimento e a main; isso não escolhe o papel dos executáveis normais.

Ordem de leitura: peer.c e superpeer.c; depois file_client.c, storage.c e directory.c; por fim os módulos de protocolo e rede. Os headers .h apresentam os tipos e contratos usados entre esses arquivos.

Demonstração e fala de apresentação

Os comandos abaixo usam o Peer B como executor e A como armazenamento. Execute a partir da raiz do projeto, com portas livres.

Preparar os serviços em terminais separados

```
make all
```

```
./bin/superpeer --port 55101 --name superpeer
```

```
./bin/peer serve 55102 127.0.0.1 55101
```

```
./bin/peer serve 55103 127.0.0.1 55101
```

Espere a inicialização e o JOIN. Escolha um arquivo.pdf existente com mais de 4 MiB para observar vários chunks. Os caminhos são locais ao computador onde o Peer executor está rodando.

Enviar a A e baixar usando B

```
./bin/peer --local-peer-port 55103 upload arquivo.pdf 127.0.0.1 55102
```

```
./bin/peer --local-peer-port 55103 download arquivo.pdf copia.pdf 127.0.0.1 55101
```

```
cmp arquivo.pdf copia.pdf
```

No upload, mostre ObjectID, hashes, quantidade de chunks e bytes comprimidos. No download, mostre as tentativas de endpoints e SHA-256 verified. cmp retorna status zero e não imprime diferença quando os bytes são iguais.

Uma fala para conectar os componentes

“Minha parte transforma a comunicação TCP em transferência de documentos. O Peer ativo calcula o ObjectID, divide em chunks de 4 MiB, calcula hashes e comprime cada parte com LZ4. Os workers enviam ao armazenamento, que verifica, persiste e publica o objeto. Depois o Peer anuncia ao Super Peer. Para baixar, o executor consulta o catálogo, busca as partes diretamente nos Peers, verifica e remonta o documento.”

O que afirmar sobre testes e escopo

make test-c2 exercita protocolo e armazenamento local. script_testes_c2.sh e tests/c2/integration.py verificam transferências entre processos. Eles têm papéis diferentes. Nesta revisão do PDF foi feita leitura do código e conferência do documento, sem nova execução das suítes funcionais.

O índice do Super Peer é volátil; restart somente dele não solicita anúncios automáticos dos Peers ativos. Nomes alternativos para o mesmo conteúdo não são aliases. Chord, Gossip, eleição, replicação automática, 2PC e garantias globais de disponibilidade continuam fora do C2 implementado.

Base da revisão: Makefile; peer.c; superpeer.c; file_client.c; local_control.c; storage.c; metadata.h/.c; directory.c; protocolos; rede; compressão; identidade/configuração; contrato de integração. Versão 9dbe2c5, consultada em 29/09/2026.